```
        }

        /* Last thing that main ( ) pthread_exit should do */
        (NULL) ;

}
```

**EXAMPLE 07**

```
# include     <pthread . h>
# include <s t d i o . h> #
include   < s t d l i b . h>
char *message ;
struct      thread_data
{
     pthread_t *thread_id ; char
    *message ;
} ;
struct      thread_data     thread_data ;

void * thr_fn ( void *threadarg ) { struct thread_data
            *my_data ;
              my_data =    ( struct        thread_data *) threadarg ;
            pthread_t * taskid          = my_data->thread_id ; char
            *hello_msg= my_data->message ;
            printf ( " Thread id is : %ld \nMessage : %s " , *taskid , hello_msg ) ; pthread_exit ( ( void *) "
            Successfull " ) ;
}

int    main( int    argc ,    char *argv [ ] ) {
              pthread_t     thread_id1 ;
              message =     "Have you    learn    something  today  ?\n" ;
            thread_data . thread_id =&thread_id1 ; thread_data . message = message ; printf ( " Creating
            thread\n" ) ; pthread_create(&thread_id1 , NULL, thr_fn , ( void *) &thread_data ) ;
            void * status ;
            pthread_join ( thread_id1 ,& status ) ; printf ( " Thread Exited with status : %s\n"
            ,( char *) status ) ; exit ( 0 ) ;
}
```



# Lab Manual 09 Thread Attributes

## 74 THREAD STATES

1. Joinable Threads

2. Detached Threads

1. When a thread is created, one of its attributes defines whether it is joinable or detached.

2. Only threads that are created as joinable can be joined. If a thread is created as detached, it can never be joined.

3. When we create a thread with default attributes a joinable thread is created.

## 76 PTHREAD FUNCTIONS

**PTHREAD_SELF()** Return the thread ID of currently executing thread.

```
pthread_t pthread_self ( void ) ;
```

Returns the unique thread ID of the calling thread.

**PTHREAD_EXIT()**
Used to terminate threads

```
pthread_exit (* status )
```

This is also the mechanism used to get a return value from a thread. Several ways of termination:

1. The thread returns from its starting routine (the main routine for the initial thread).

2. The thread makes a call to the pthread_exit subroutine.

3. The thread is canceled by another thread via the pthread_cancel routine.

4. The entire process (main) is terminated due to a call to the exit subroutines.

## 77 JOINING THREADS

```
int     pthread_join        ( pthread_t tid ,     void ** status ) ;
```

1. "Joining" is one way to accomplish synchronization between threads.

2. The pthread_join() subroutine blocks the calling thread until the specified threadid thread terminates.

3. The programmer is able to obtain the target thread's termination return status if it was specified in the target thread's call to pthread_exit().

4. A joining thread can match one pthread_join() call. It is a logical error to attempt multiple joins on the same thread.

```
                <s t d i o . h>
# include < s t d l i b . h> # include <pthread . h>

void * thr_fn1 ( void *arg ) { printf ( " Thread 1 exiting\n" ) ;
            void * status=" Success " ;
            pthread_exit ( ( void *) status ) ;
}

void * thr_fn2 ( void *arg ) { printf ( " Thread 2 exiting\n" ) ;
            pthread_exit ( ( void * ) 2 ) ;
}

int main( void ) { pthread_t tid1 , tid2 ; pthread_create(&tid1 , NULL, thr_fn1 , NULL) ; pthread_create(& tid2 ,
            NULL, thr_fn2 , NULL) ;




            void * tret1 ; void *
            tret2 ;
```

**EXAMPLE 0**

```
# include
```

```
/*          why passing      adress    of        pointer   tret1     ? see     Pthread_join  prototype ! !     */
            pthread_join ( tid1 , &tret1 ) ; pthread_join ( tid2 ,
            &tret2 ) ;
/*          casting      tret1     to char*                                                                      */
            printf ( " Thread        1 exit    code        is : %s\n" ,  ( char *) tret1 ) ;

                                                                                    printf ( " Thread
                    2 exit     code      is : %ld\n" ,  ( long ) tret2 ) ;

            exit ( 0 ) ;
}
```

## 78 Thread Detaching

1. By default threads are created joinable.

2. Instead of waiting for a thread, the executing thread can specify that

   (a) It does not require a return value.

   (b) Or any explicit synchronization with that thread.

3. The pthread_detach() routine can be used to explicitly detach a thread even though it was created as joinable.

4. After this call, no thread can wait for detached thread and it executes independently until termination.

```
int    pthread_detach ( pthread_t          tid ) ;
```

5. There is no converse routine.

6. Threads can detach themselves by calling pthread_detach with an argument of pthread_self().

**EXAMPLE 01**

```
# include      <s t d i o . h>
# include < s t d l i b . h> # include <pthread . h>
void * thread_function ( void *arg ) ; char message [ ] = "iam thread 1 " ;
int main ( ) { pthread_t tid ; pthread_create(&tid ,NULL, thread_function , ( void *) message ) ; // In 2nd run
               make following statement comment then see pthread_detach ( tid ) ;
                int   j o i n r e t ;
               j o i n r e t =pthread_join ( tid ,NULL) ;

               if ( j o i n r e t ==0) //when join successfull printf ( " join was successfull : The main thread was
                                waiting for thread 1\n" ) ;
```

```
               else printf ( " Join failed : The main thread is not waiting for thread 1\n" ) ;
               printf ( "The main thread finished , bye!\n" ) ; pthread_exit (NULL) ;


}*

void

            thread_function ( void *arg ) {
              sleep ( 2 ) ;
printf ( " thread_function                          is running . Argument was\" %s\"\n" , ( char *)
                                                    arg ) ;
}printf ( " thread 1 awaked from sleep , and exiting now\n" ) ; pthread_exit (NULL) ;
```

**EXAMPLE 02**

```
# include      <s t d i o . h>
# include < s t d l i b . h> #
include <pthread . h>
void * thread_function1 ( void *arg ) ; void * thread_function2 ( void *arg ) ; char message1 [ ] =
"iam thread 1 " ; char message2 [ ] = "iam thread 2 " ; pthread_t tid1 , tid2 ;
int  main  ( )    { pthread_create(&tid1 ,NULL,  thread_function1    , ( void *) message1 )        ;
            pthread_create(&tid2 ,NULL, thread_function2 , ( void *) message2 ) ; pthread_detach (
            tid1 ) ;
             int   j o i n r e t ;
            j o i n r e t =pthread_join ( tid1 ,NULL) ;

             //when          join       successfull if ( j o i n r e t ==0) printf ( " join       was          successfull :
            The main          thread    was       waiting   for        thread else
                  printf ( " Join       failed : The main      thread    is  not  waiting       for       thread  1

            pthread_join ( tid2 ,NULL) ; printf ( "The main received exit status from tid2 ,now exiting , bye!\n" ) ;
            pthread_exit (NULL) ;
}

void * thread_function1 ( void *arg ) { sleep ( 2 ) ; printf ( " thread_function 1 is running . Argument was %s\n"
            , ( char *) sleep ( 5 ) ; printf ( " thread 1 awaked from sleep , and exiting now\n" ) ; pthread_exit
            (NULL) ;
}

void * thread_function2 ( void *arg ) {
            sleep ( 1 ) ; int j o
            i n r e t ;
            j o i n r e t =pthread_join ( tid1 ,NULL) ;
```

```
            if ( j o i n r e t ==0) //when join successfull printf ( " join was successfull : The thread 2 was waiting
            for thread 1 else
                  printf ( " join failed : The thread 2 is not waiting for thread 1\n" ) ; printf ( " thread_function 2
            is running . Argument was %s\n" , ( char *) printf ( " thread 2 is exiting now\n" ) ; pthread_exit (NULL)
            ;
}
```

```
1\n" ) ;
\n" ) ;
arg ) ;
\n" ) ;
arg ) ;
```

## 79 Thread Attributes

The pthread interface allows us to fine-tune the behavior of threads using the pthread_attr_t structure by modifying the default attributes.

1. An initialization function exists to set the attributes to their default values.
2. Another function exists to destroy the attributes object. If the initialization function allocated any resources associated with the attributes object, the destroy function frees those resources.
3. Each attribute has a function to get the value of the attribute from the attribute object.
4. Each attribute has a function to set the value of the attribute. In this case, the value is passed as an argument, by value.

## 80 Thread Detaching Using Attributes

To explicitly create a thread as joinable or detached, the attr argument in the pthread_create() routine is used. The typical 4 step process is:

1. Declare a pthread attribute variable of the pthread_attr_t data type.
2. Initialize the attribute variable with pthread_attr_init().
3. Set the attribute detached status with pthread_attr_setdetachstate().
4. When done, free library resources used by the attribute with pthread_attr_destroy().

### Important Points

- A detach thread can also be created using thread attributes.

```
pthread_attr_t          thread_attr ;
```

- Calling pthread_attr_init, the pthread_attr_t structure contains the default values for all the thread attributes supported by the implementation. pthread_attr_init (& thread_attr ) ;
- detachedstate: This attribute allows us to avoid the need for threads to rejoin.

```
int     pthread_attr_setdetachstate ( pthread_attr_t                ∗ attr ,     int          detachstate ) ;
```

- The two possible flag values for pthread_attr_setdetachstate are PTHREAD_CREATE_JOINABLE and PTHREAD_CREATE_DETACHED.

## 81 pthread_attr_destroy()

```
pthread_attr_destroy ( pthread_attr_t                ∗ a t t r )
```

1. To deinitialize a pthread_attr_t structure, we call pthread_attr_destroy.




2. If an implementation of pthread_attr_init allocated any dynamic memory for the attribute object, pthread_attr_destroy will free that memory

3. pthread_attr_destroy will initialize the attribute object with invalid values, so if it is used by mistake, pthread_create will return an error code.

**EXAMPLE 03**

1\n" ) ;




```
# include      <s t d i o . h>
# include < s t d l i b . h> #
include <pthread . h>
void ∗ thread_function ( void ∗arg ) ; char message [ ] =
"Iam thread 1 " ;
int main ( ) { pthread_t tid ; pthread_attr_t
               thread  attr ;
               pthread  attr  init (& thread  attr ) ;
               //comment          following line in 2nd run pthread  attr  setdetachstate (&thread  attr
               ,PTHREAD  CREATE  DETACHED) ;
                //uncomment       following          line in 2nd run
               //pthread_attr_setdetachstate (&thread_attr , PTHREAD_CREATE_JOINABLE ) ; pthread_create(&tid
               ,& thread_attr , thread_function , ( void ∗) message ) ; pthread_attr_destroy(& thread_attr ) ; int j o i
               n r e t ;
               j o i n r e t =pthread_join ( tid ,NULL) ;

               if ( j o i n r e t ==0) //when join successfull printf ( " join was successfull : The main thread was waiting
               for thread else
                     printf ( " Join failed : The main thread is not waiting for thread 1 printf ( "The main thread
               finished , bye!\n" ) ; pthread_exit (NULL) ;
}

void ∗ thread_function ( void ∗arg ) { printf ( " thread_function is running . Argument was\" %s\"\n" , ( char ∗)
               sleep ( 4 ) ; printf ( " thread awaked from sleep , and exiting now\n" ) ; pthread_exit (NULL) ;
}
```

\n" ) ;

arg ) ;

## 82 Some Other Thread Attributes

```
int  pthread_attr_setdetachstate ( pthread_attr_t * attr ,                    int       detachstate ) ;

int  pthread_attr_setschedpolicy ( pthread_attr_t * attr ,                    int      policy ) ;
                                    int  pthread_attr_setscope ( pthread_attr_t * attr ,    int          scope ) ;

                    int  pthread_attr_getscope ( const          pthread_attr_t * attr ,         int *scope ) ;

int  pthread_attr_setstacksize ( pthread_attr_t                              * attr ,    int                    scope ) ;

int  pthread_attr_getstacksize ( const  pthread_attr_t * attr , int *scope ) ;
```

## 83 Thread Cancelation

1. Sometimes, we want one thread to be able to ask another thread to terminate, rather like sending it a signal. There is a way to do this with threads using pthread_cancel() system call.




2. One thread can request that another in the same process be canceled by calling the pthread_cancel function.
```
pthread_cancel ( pthread_t
                 tid )
```
3. In the default circumstances, pthread_cancel will cause the thread specified by **tid** to behave as if it had called **pthread_exit** with an argument of PTHREAD_CANCELED.

## 84 Thread Cancelation Options

A thread can change its cancelability state by calling pthread_setcancelstate.

```
pthread_setcancelstate ( int          state ,    int * oldstate ) ;
```

The first parameter is either:

1. **PTHREAD_CANCEL_ENABLE** which allows it to receive cancel requests.
2. **PTHREAD_CANCEL_DISABLE** which causes them to be ignored. The oldstate pointer allows the previous state to be retrieved. If you are not interested, you can simply pass NULL.

### Important Points

- A thread starts with a default cancelability state of PTHREAD_CANCEL_ENABLE.
- When the state is set to PTHREAD_CANCEL_DISABLE, a call to pthread_cancel will not kill the thread.

- The cancellation request remains pending for the thread.When the state is enabled again, the thread will act on any pending cancellation requests at the next cancellation point.

## 85 THREAD CANCELATION TYPES

If cancel requests are accepted, there is a second level of control the thread can take, the cancel type, which is set with pthread_setcanceltype.

```
int  pthread_setcanceltype ( int  type , int *oldtype ) ;
```

The type can take one of two values:

1. **PTHREAD_CANCEL_ASYNCHRONOUS** which causes cancellation requests to be acted upon immediately.

2. **PTHREAD_CANCEL_DEFERRED** which makes cancellation requests wait until the thread executes one of these functions: pthread_join, pthread_cond_wait, pthread_cond_timedwait, sem_wait etc.

By default, threads start with the *cancellation state* as **PTHREAD_CANCEL_ENABLE** and the *cancellation type* as **PTHREAD_CANCEL_DEFERRED**. **EXAMPLE 04**

```
# include       <pthread . h>
# include <s t d i o . h> #
include < s t d l i b . h>
void *thread  function1 ( void *arg ) ; pthread  t tid1 ;
      int main ( ) { pthread_create(&tid1 , NULL, thread_function1 , NULL) ; sleep ( 2 ) ;
    printf ( "Main thread : Canceling thread 1 . . . \ n" ) ; pthread_cancel ( tid1 ) ; printf
            ( "Main thread : exiting\n" ) ;
            pthread_exit (NULL) ;
}

void * thread  function1 ( void *arg ) {
             int       i , oldtype , oldstate ;
             // comment       following          line in 2nd run
            pthread_setcancelstate (PTHREAD_CANCEL_DISABLE,& oldstate ) ; // uncomment
            following line in 2nd run
            //pthread_setcancelstate (PTHREAD_CANCEL_ENABLE,&oldtype ) ; printf ( " Thread 1: iam running an
            old cancel state was %d\n" , oldstate ) ; for ( i = 0; i < 10; i ++) {
                  printf ( " Thread 1: iam s t i l l running (%d ) . . . \ n" , i ) ; sleep ( 2 ) ;
            }
            pthread_exit ( 0 ) ;
}
```

**EXAMPLE 05**

```
# include       <pthread . h>
# include <s t d i o . h> #
include < s t d l i b . h>
void *thread_function1 ( void *arg ) ;
```

```
void *thread_function2 ( void *arg ) ; pthread_t tid1 , tid2 ;
int      main  (  )  {  pthread_create(&tid1  ,  NULL,  thread_function1  ,  NULL)  ;
      pthread_create(&tid2 , NULL, thread_function2 , NULL) ; sleep ( 1 ) ; printf (
      "Main thread : Canceling thread 1 . . . \ n" ) ; pthread_cancel ( tid1 ) ; printf (
      "Main thread : exiting\n" ) ;
            pthread_exit (NULL) ;
}

void * thread_function1 ( void *arg ) {
            int i , oldstate , oldtype ; ; pthread_setcancelstate (PTHREAD_CANCEL_ENABLE,&
            oldstate ) ; pthread_setcanceltype (PTHREAD_CANCEL_DEFERRED,&oldtype ) ;
            //pthread_setcanceltype (PTHREAD_CANCEL_ASYNCHRONOUS,&oldtype ) ;
            //sleep ( 1 ) ; printf ( " Thread 1:my old cancel state was %d and old cancel type was %d\n" , oldstate
            , oldtype ) ;
             for ( i  =  0;  i       <    5;  i ++)     {
            printf ( " Thread 1: iam running (%d)\n" , i ) ; if ( i =1){
              printf ( " Thread 1: iam waiting for thread 2 to finish\n" ) ; pthread_join ( tid2 , NULL) ; }
            }
               // Following       statement            will  never  run ! ! ! ! !
            printf ( " Thread 1: Thread 1 exited , now me aswell\n" ) ; pthread_exit ( 0 ) ;
}

void * thread_function2 ( void *arg ) {
             int  i ;
            sleep ( 2 ) ;
             for ( i  =  0;  i       <    5;  i ++)     {
                  printf ( " Thread      2: iam          running (%d)\n" , i ) ;
                printf ( " Thread 2: thread 1 received cancel signal but is waiting for me to finish\n" ) ;
            }
              printf ( " Thread      2: iam      exiting\n" ) ;
            pthread_exit ( 0 ) ;
 }
```